13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define INPUT_SIZE 200

int main() {

    char input[INPUT_SIZE];
    char filename[INPUT_SIZE];

    int saved_stdin;
    int file_fd;

    printf("Input Redirection System\n");
    printf("-----------------------\n");
    printf("Enter input redirection in the form: < filename\n\n");

    printf("shell> ");
    fflush(stdout);

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Check for input redirection */
    if (input[0] != '<') {

        printf("Error: Input redirection '<' not found.\n");
        return 1;
    }

    /* Extract filename */
    if (sscanf(input + 1, "%199s", filename) != 1) {

        printf("Error: Missing input file.\n");
        return 1;
    }

    printf("Input file: %s\n", filename);

    /* Open file for reading */
    file_fd = open(filename, O_RDONLY);

    if (file_fd == -1) {

        perror("Error opening input file");
        return 1;
```

```
  GNU nano 8.7.1
        perror("dup");
        close(file_fd);
        return 1;
    }

    /*
     * Redirect stdin from the file.
     */
    if (dup2(file_fd, STDIN_FILENO) == -1) {

        perror("dup2");
        close(file_fd);
        close(saved_stdin);
        return 1;
    }

    close(file_fd);

    printf("Standard input redirected from file.\n");

    /*
     * Read data from redirected stdin.
     */
    printf("\nFile Contents:\n");
    printf("--------------------------\n");

    char line[INPUT_SIZE];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        printf("%s", line);
    }

    printf("\n--------------------------\n");

    /*
     * Restore original stdin.
     */
    if (dup2(saved_stdin, STDIN_FILENO) == -1) {

        perror("Failed to restore stdin");
        close(saved_stdin);
        return 1;
    }

    close(saved_stdin);

    printf("Standard input restored successfully.\n");
    printf("Input redirection completed successfully.\n");

    return 0;
}
```

```
sanjana3027@DESKTOP-QJF5AGU:
 GNU nano 8.7.1
Hello Copilot
This is input redirection.
OSSP Skill 13
File input is working correctly.
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano Skill13a.c
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano input.txt
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ gcc Skill13a.c -o s13a
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ ./s13a
Input Redirection System
------------------------
Enter input redirection in the form: < filename

shell> < input.txt
Input file: input.txt
File opened successfully.
Standard input redirected from file.

File Contents:
------------------------
Hello Copilot
This is input redirection.
OSSP Skill 13
File input is working correctly.

------------------------
Standard input restored successfully.
Input redirection completed successfully.
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$
```

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
 GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 200

int main() {

    char input[INPUT_SIZE];
    char filename[INPUT_SIZE];

    int file_fd;
    int saved_stdout;

    printf("Output Redirection\n");
    printf("------------------\n");

    printf("Enter output redirection: ");
    fflush(stdout);

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Check for > symbol */
    char *redirect = strchr(input, '>');

    if (redirect == NULL) {
        printf("Error: '>' not found.\n");
        return 1;
    }

    /* Get filename */
    if (sscanf(redirect + 1, "%199s", filename) != 1) {
        printf("Error: Output filename missing.\n");
        return 1;
    }

    printf("Output file: %s\n", filename);

    /* Create output file */
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

    if (file_fd == -1) {
        perror("Error opening file");
        return 1;
    }
```

```
GNU nano 8.7.1
        return 1;
    }

    printf("Output file created successfully.\n");

    /* Save original stdout */
    saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1) {
        perror("dup");
        close(file_fd);
        return 1;
    }

    /* Redirect stdout to file */
    if (dup2(file_fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(file_fd);
        close(saved_stdout);
        return 1;
    }

    close(file_fd);

    /* These lines go into the file */
    printf("Hello from output redirection!\n");
    printf("This data is redirected to the file.\n");
    printf("Skill 14 is working successfully.\n");

    fflush(stdout);

    /* Restore stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Failed to restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Standard output restored successfully.\n");

    /* Verify output file */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nVerified Output Data:\n");
    printf("----------------------\n");
```

```
    close(saved_stdout);

    printf("Standard output restored successfully.\n");

    /* Verify output file */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nVerified Output Data:\n");
    printf("----------------------\n");

    char line[INPUT_SIZE];

    while (fgets(line, sizeof(line), file) != NULL) {
        printf("%s", line);
    }

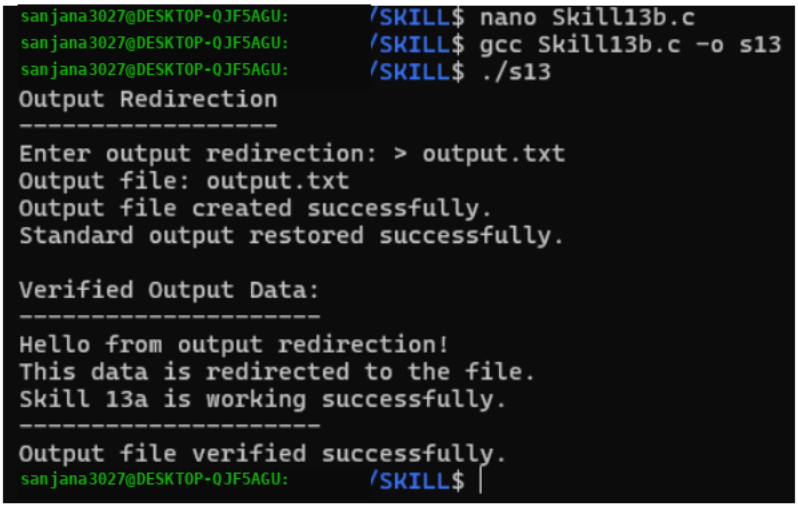
    printf("----------------------\n");

    fclose(file);

    printf("Output file verified successfully.\n");

    return 0;
}
```

Result:


sanjana3027@DESKTOP-QJF5AGU: /SKILL$ nano Skill13b.c
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ gcc Skill13b.c -o s13
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ ./s13
Output Redirection
Enter output redirection: > output.txt
Output file: output.txt
Output file created successfully.
Standard output restored successfully.
Verified Output Data:
Hello from output redirection!
This data is redirected to the file.
Skill 13a is working successfully.
Output file verified successfully.
sanjana3027@DESKTOP-QJF5AGU: /SKILL$